

论云原生架构及其应用

近年来，随着数字化转型不断深入，科技创新与业务发展不断融合，各行各业正在从大工业时代的固化范式进化成面向创新型组织与灵活型业务的崭新模式。在这一背景下，以容器和微服务架构为代表的云原生技术作为云计算服务的新模式，已经逐渐成为企业持续发展的主流选择。云原生架构是基于云原生技术的一组架构原则和设计模式的集合，旨在将云应用中的非业务代码部分进行最大化剥离，从而让云设施接管应用中原有的大量非功能特性（如弹性、韧性、安全、可观测性、灰度等），使业务不再有非功能性业务中断困扰的同时，具备轻量、敏捷、高度自动化的特点。云原生架构有利于各组织在公有云、私有云和混合云等新型动态环境中，构建和运行可弹性扩展的应用，其代表技术包括容器、服务网格、微服务、不可变基础设施和声明式 API 等。

请围绕“论云原生架构及其应用”论题，依次从以下三个方面进行论述：

- 1.概要叙述你参与管理和开发的软件项目以及承担的主要工作。
- 2.服务化、弹性、可观测性和自动化是云原生架构的四类设计原则，请简要对这四类设计原则的内涵进行阐述。
- 3.具体阐述你参与管理和开发的项目是如何采用云原生架构的，并且围绕上述四类设计原则，详细论述在项目设计与实现过程中遇到了哪些实际问题，是如何解决的。

范例

摘要部分：

年月，我作为技术负责人参与了某市**平台建设项目，该项目主要包括**，打造**，构建**。本文将以**建设为例，阐述云原生架构在本项目中具体实践。基于云原生弹性原则，解决以往项目中系统资源无法根据需求进行弹性缩扩容的问题。基于云原生自动化原则，解决项目中应用上线周期长，复杂度高的问题。基于云原生服务化原则，解决以往系统建设中各系统共性功能重复建设的问题。通过云原生架构的成功使用，平台实现资源层和应用层的弹性伸缩能力，提升了平台对外提供服务的能力，及项目整体自动化水平。现在平台已经顺利上线运营，取得非常良好社会效益和经济效益。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写 280 到 300 字】

正文部分：

年月公司中标我市**项目，项目总投资**万元人民币，项目建设周期**年，我作为技术负责人全程参与了本项目。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

本项目**，项目总体建设内容包括***，为***提供***；采用***，我们基于****技术路线，将平台****。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 字左右】

我们在项目中采用云原生架构，充分体现云原生架构的自动化、服务化、弹性，及可观测性四大原则。自动化原则，体现应用的自动发布，自动化代码检查，及自动化测试等自动流程；服务化原则，体现在云原生架构实现了“微服务，轻应用”，按需对外提供服务；弹性原则主要来源于云计算特有的对云资源的弹性伸缩能力；可观测性，通过相关采集系统和监测工具的使用，系统的可观测性进一步加强。接下来，我将基于本项目的实践着重论述自动化原则，服务化原则及弹性原则。

一、自动化原则。以往的项目中上线一个应用，需要准备环境，搭建服务器，有时因为开发时没考虑上线时的需要，如浏览器及客户端等适配问题，经常导致上线工作周期长，问题多，经常返工。本项目中我们一方面采用了 K8S 来进行容器的编排和镜像的管理，基于容器技术实现了应用的自动化部署，及按需灵活缩扩容。为了实现自动化上线和自动运维，我在项目倡导了 DevOps 实践，开发人员从写下第一行代码开始，就要考虑相关后期上线及运维的工作，运维人员也在项目一开始就介入其中，使得从编码到上线更加顺畅高效。同时我也引入了相关自动代码检查，代码安全漏洞扫描工具，代替以往的人工走查和桌面检查，我们的测试团队也开发了相关自动测试工具，引入相关云服务商提供的自动压测工具，极大提升了团队的工作效率和项目质量。

二、服务化原则。以往系统建设中，多是竖井式建设，烟囱式系统，小而全，大而全，但工业互联网平台将要开发大量的工业应用，如果还是按照以前方式，一是工作量巨大，二是一定会有大量的重复建设，造成工期和成本的巨大浪费。项目中我们基于 SOA 的架构思想，采用现在流行的“微服务，轻应用”理念，采用 spring cloud+docker 的技术路线，我们将平台沉淀的相关设备管理、生产管理、供应链管理，及工业大数据分析能力拆分解耦，封装为一个一个微服务，实现平台整体的“高内聚，低耦合”，这些服务通过微服务网关支撑各类上层工业应用服务建设及工业 app 开发。通过这种方式，我们一方面实现了共性能力的复用，达到了集约化建设目标，另一方面也将平台能力通过微服务的方式开放给第三方开发者，将平台沉淀的相关数据分析，智能制造，设备管理能力对外赋能，为构建开放共享的区域工业互联网应用生态打下了坚实基础。

三、弹性原则。以往的系统平台建设中，如何按需对项目所需要的存储、计算、网络资源进行动态缩扩容，以及在面临海量高并发的情况下，如何能够实现应用服务的高可用性也是一直没有很好解决的难题。我通过打造两方面的能力，实现在云原生架构的区域工业互联网平台的弹性伸缩能力。为了解决这个问题，我们从三方面着手：一是在 IaaS 层通过 VMware 对存储，计算资源进行了虚拟化，构建了统一的虚拟化云资源池，并基于 OpenStack 打造了云资源服务统一管理平台，也就是区域工业云平台，基于 OpenStack 实现了云资源的弹性供给，动态管理及按需缩扩容。二是在 PaaS 层通过 K8S，实现根据服务请求数量，实时调整相关微服务数量，使用系统就算在高并发条件下也具有非常高的可用性，比如在实际使用中我们设备平台的协议转换服务请求较多时，通过 k8s 及时上线相关备用服务，满足忙时业务需求，当闲时也可以非常方便的下线相关应用服务，释放系统资源，提升了平台服务层的弹性。

年月项目正式上线运营，目前平台已经为区域内 50 多家工业企业提供上云服务，接入工业设备超 200 台，沉淀工业大数据近 50PB，提供工业微服务近 200 个，打造了工业 app 近 30 个，初步构建了开放共享的区域工业互联网生态，取得了比较良好的社会效益和经济效益。

通过项目的实践，我深刻体会到云原生架构不仅是技术创新，更是组织与管理的变革，如果不能建立起开发、运维、及质量保证融合推进的组织架构，没有敏捷开发及快速迭代的文化，云原生架构带来自动化，服务化，弹性，及可观测性优势都会大打折扣，流于形式。虽然项目中取得一定成绩，但也暴露了比较多的问题，项目初期由于大量采用开源的新技术和新架构，技术复杂开发难度大，导致了项目推进举步维艰，一方面我引进外部专家，开展多轮全员培训，并招聘相关专业技术人员进入团队；另一方面根据我们项目的实际情况，提出直接按需购买相关云厂商产品服务的方案。通过综合施策，项目得以顺利开展。项目的成功不是我一个人的功劳，在此向参与项目各位成员表示最真诚地感谢。凡是过往，皆为序章，我将继续保持空杯心态，不断学习提升自己的综合素养，继续为祖国的信息化事业贡献自己的绵薄之力。



论大规模分布式系统缓存设计策略

大规模分布式系统通常需要利用缓存技术减轻服务器负载、降低网络拥塞、增强系统可扩展性。缓存技术的基本思想是将客户最近经常访问的内容在缓存服务器中存放一个副本，当该内容下次被访问时，不必建立新的数据请求，而是直接由缓存提供。良好的缓存设计，是一个大规模分布式系统能够正常、高效运行的必要前提。在进行大规模分布式系统开发时，必须从一开始就针对应用需求和场景对系统的缓存机制进行全面考虑，设计一个可伸缩的系统缓存架构。

请围绕“大规模分布式系统缓存设计策略”论题，依次从以下三个方面进行论述。

- 1.概要叙述你参与实施的大规模分布式系统开发项目以及你所担任的主要工作。
- 2.从不同的用途和应用场景考虑，请详细阐述至少两种常见的缓存工作模式，并说明每种工作模式的适应场景。
- 3.阐述你在设计大规模分布式系统的缓存机制时遇到了哪些问题，如何解决。

范例

摘要部分：

年月，我单位联合某高校研发了**系统。系统以**功能为核心，分为**模块等，支持对接教务平台。在项目中我担任系统架构师，主要负责架构设计工作。本文以该平台为例，主要论述了大规模分布式系统缓存设计策略在项目中的具体应用。系统缓存基于 Redis 内存数据库实现，工作模式的选择上根据不同数据类型，采用了主从模式与集群模式结合的设计；通过数据持久化、数据备份计划、冗余机制和监控平台等方法实现高可用性；通过数据访问层封装同步操作实现缓存一致性，通过哈希环实现分布式算法。最终系统顺利上线，受到用户一致好评。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写 280 到 300 字】

正文部分：

我在**单位任职，过往成果有**等。**年**月，我单位联合某大学研发了**项目（以下简称为“OJ 系统”），以取代原有传统的编程上机考试平台。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

系统以**功能为核心，主要分为**等模块。**模块主要负责**；**模块用于**；**模块用于**。在这个项目中，我担任了系统架构师的职务，主要负责系统的架构设计相关工作。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 字左右】

系统需要支撑全校学生编程课程的学习考试以及大量校外用户的使用，为保证系统性能和用户体验，采用缓存技术进行优化尤为重要。常见的缓存分为三种工作模式：单点模式、主从模式和集群模式。单点模式各个应用服务共享同一个缓存实例，主要瓶颈在于缓存服务器的内存大小、处理能力和网络带宽，适合缓存要求简单、数据量和吞吐量小、性能要求低的场景；主从模式将缓存的数据复制到多台机器上，数据彼此同步，可分散压力、获得更高的吞吐能力，适合数据量不大、改动频率不大、性能要求高的场景；集群模式节点间共享数据，部分冗余保证一定程度的可用性，可存储超过单个服务内存容量数据，通过增加服务器缓解数据增长和压力增加，适合总体数据量大、可伸缩性需求高、客户端数量庞大的场景。

OJ 系统使用 Spring Cloud 微服务架构开发，基于 Redis 内存数据库实现缓存，运用了多种缓存设计策略，本文着重从工作模式的选择、高可用性的设计、缓存一致性与分布式算法三个方面的问题展开论述。

1. 工作模式的选择

OJ 系统需要支撑全校学生编程课程的学习考试，以及大量校外用户的使用，对可靠性和性能要求高，因此单点模式不予考虑。经过对 OJ 系统业务需求的分析，我们归纳了三种需要缓存的数据类型。第一种是系统配置项、角色权限分配等元数据信息。这部分信息使用频率很高但数据量小，不会经常改动，我们采用主从模式来缓存，主缓存节点供系统配置模块写入数据，从缓存节点分别供具体业务服务调用。在系统启动时，自动将相关数据装入缓存。信息修改时，先写入主缓存节点，再同步更新至从缓存节点。第二种是用户登录的会话状态，这部分信息使用频率高、更新频繁，且几乎所有的业务服务都需要依赖此类数据，为保证性能与可靠性，我们采用集群方式来缓存，集中管理用户会话，各个业务服务通过数据访问层来调用具体的缓存节点。第三种是业务逻辑中一定时间内不会变化，但数据量大的信息，如试题信息、实验作业信息等，以及统计机制自动识别的高频访问信息，也采用集群方式缓存。

2. 高可用性的设计

为实现 OJ 系统缓存的高可用性，一方面我们开启了 Redis 数据持久化功能，并配置了相应的备份策略，能够有效地解决数据误操作和数据异常丢失的问题，另一方面，我们设计了一些冗余机制。在主从模式中，采用多机主备架构实现冗余，在主节点故障时，自动进行主备切换，将从节点提升为主节点继续服务，保证服务的平稳运行。如果主节点和从节点之间连接断开，重新连接时从节点会进行数据的部分重同步，当无法进行部分重同步时，会进行全量重同步。在集群模式中，采用 Cluster 技术实现冗余，每个节点保存数据和整个集群状态，负责一部分哈希槽，每个节点都和其他所有节点连接并共享数据。为了使在部分节点失效或者大部分节点无法通信的情况下集群依然可用，在集群内部使用了主从复制模型，每个节点都会相应地有 1~N 个从节点，当某个节点不可用时，集群便会将它的从节点提升作为新的主节点继续服务。缓存服务发生异常时，可通过 OJ 系统的服务监控平台产生报警，提醒运维人员及时处理。

3. 缓存一致性与分布式算法

为保证 Redis 缓存与原数据库的数据一致性，在读取数据时，系统先读取 Redis 缓存中的数据；若缓存中不存在所需数据，则从原数据库读取并同步写入 Redis 缓存。在写回或删除数据时，将数据写入原数据库，并同步淘汰 Redis 缓存中的数据。业务服务通过使用专门的数据访问层来调用增加缓存后的数据库，使数据缓存机制对应用透明。在缓存内部实现一致性，通过分布式哈希算法来实现，为考虑到日后缓存集群的扩展需要，因此不能使用简单的模 N 哈希法，我们在 OJ 系统中采用了哈希环的算法。该算法构造一个长度为 232 的整数环，将 Redis 节点放置于环上，当业务服务调用缓存时，首先以服务的应用 ID 作为键值计算哈希在环上定位，然后沿顺时针方向找到最近的 Redis 节点，完成映射。当缓存服务集群中有节点故障，以及添加新节点时，只会影响上一个节点的数据，避免了发生缓存雪崩的情况，提高了容错性和扩展性。哈希环中缓存节点过少时易发生缓存倾斜，我们通过增加虚拟节点的方式解决了该问题。

我们在这次系统设计中，还使用了很多其他的缓存设计策略，这里不再一一赘述。系统在经过性能测试、负载测试、压力测试、稳定性测试后，自**年**月正式上线已运行一年有余，在学校的日常教学考试和竞赛培训中投入使用，截至目前已有 3000 名以上的学生用户、评测了 70000 份以上的程序代码，获得了单位同事领导和学校教师们的一致好评。日常使用过程中最高出现过 600 余用户同时在线进行实验作业提交、评测的情况，基本未出现页面加载缓慢、请求超时的问题，满足了高校编程课平台的基本性能需求。

实践证明，OJ 系统项目能够顺利上线，并且稳定运行，与系统采用了合理的缓存设计密不可分。经过这次大规模分布式系统缓存设计的方法和实施的效果后，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系统在各方面的设计，使整个系统能够更加好用，更有效地服务于高校师生。

高可靠性系统中软件容错技术的应用

容错技术是当前计算机领域研究的热点之一，是提高整个系统可靠性的有效途径，许多重要行业（如航空、航天、电力、银行等）对计算机系统提出了高可靠、高可用、高安全的要求，用于保障系统的连续工作，当硬件或软件发生故障后，计算机系统能快速完成故障的定位与处理，确保系统正常工作。

对于可靠性要求高的系统，在系统设计中应充分考虑系统的容错能力，通常，在硬件配置上，采用了冗余备份的方法，以便在资源上保证系统的可靠性。在软件设计上，主要考虑对错误（故障）的过滤、定位和处理，软件的容错算法是软件系统需要解决的关键技术，也是充分发挥硬件资源效率，提高系统可靠性的关键。

请围绕“高可靠性系统中软件容错技术的应用”论题，依次从以下三个方面进行论述。

1. 简述你参与设计和开发的、与容错相关的软件项目以及你所承担的主要工作。
2. 具体论述你在设计软件时，如何考虑容错问题，采用了哪几种容错技术和方法。
3. 分析你所采用的容错方法是否达到系统的可靠性和实时性要求。

范例

摘要部分：

年月，本人在公司参加了**系统的开发。该项目主要功能是做**业务。本人在该项目中担任了系统架构师职位，负责该系统的架构设计工作。本文以该项目为例，主要论述了高可靠性系统中软件容错技术的应用。为了防止硬件和软件的单节点故障问题，我们采用了集群部署的方式来提高软硬件服务的可靠性；数据库是所有业务服务都依赖的基础服务，我们采用了主备部署的方式提高数据库的可靠性；为了提高单个业务服务软件健壮性与可靠性，我们采用了防卫式程序设计、降低复杂度设计和错误日志记录的方案来提高服务的可靠性。通过上述方案的使用，保障了服务的高可靠性，最终项目顺利上线，持续稳定运行。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写到 280 到 300 字】

正文部分：

某公司**为了应对***，因此**研发**系统。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

年月，本人在公司参加了**系统的开发。该系统的核心业务场景是**。该系统的主要功能模块有**等。**模块提供了**；**模块提供了**；**模块提供了**等。本人在该系统中担任系统架构师职位，负责该系统的架构设计工作。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 字左右】

系统可靠性包括硬件可靠性和软件可靠性，两个方面都需要我们考虑到，软件的复杂度比硬件高，所以软件系统发生故障的概率也会高于硬件系统。硬件在使用时间过长的情况下，物理退化的现象加重，故障发生概率就会变高，通常会采用主动冗余、监控检错配合报警的方案来解决。软件系统包含各种的业务逻辑，复杂度较高，故障概率高，通常我们采用防卫式程序设计、N 版本程序设计、降低复杂度设计等方案来提高单个服务的自身可靠性；单个服务的可靠性不可能达到百分百，总归避免不了特殊场景下导致服务崩溃无法提供服务支持的情况，针对这一问题通常采用集群部署配合监控报警的方案来解决，即使一台服务崩溃，还有其他服务提供者进行业务支持。合理的采用容错技术方案，可以大大提高服务的可靠性。

在系统中，我们采用了多种容错技术来提高系统的可靠性，下文着重讨论应用系统集群部署、数据库主备部署和程序设计方面在该项目中容错方面具体的应用和效果。

一、系统集群部署容错

单个硬件设备或者说单个软件服务进程，都不可避免的会发生故障导致服务中断。如果单个硬件设备上部署一个或多个软件服务进程，当硬件发生故障，所有软件服务都将不可正常服务；如果一个软件服务就部署了一个进程在提供服务，一旦这个软件进程发生故障，服务也将不可正常运行。针对上述问题，集群部署方案可以很好的解决。同一个软件服务，在不同的多台物理设备上做集群部署，即使一台物理设备发生故障或者一个软件服务进程发生故障，都有其他的多台物理设备和软件进程在提供服务，达到服务永不中断的目的。集群部署就要考虑到请求分发和状态的问题，因为我们的业务是 pos 机交易，走的是 TCP/IP 协议，采用了硬件 F5 做负载分发业务；每个业务进程都尽量做成无状态服务，实在需要处理状态的，我们采用了 redis 集群来保存状态数据的方案。发生问题的物理设备和软件服务要尽快被发现然后修复，我们采用了 zabbix 分布式服务监控系统来实现，一旦监控发现问题，立马发送邮件给相关维护人员进行处理。通过集群和监控报警的配合，大大提高了整个系统的可靠性和实时性。

二、数据库主备部署容错

数据库服务是整个收单交易系统的基础服务，每个业务模块都要使用到，所以一旦数据库出现问题，整个服务将中断，影响重大。单个数据库服务不可能说永远不可能出现故障，我们要做到的首先是提升单个数据库服务的可靠性，其次是如果单个数据库服务发生故障，立刻快速使用备用数据库继续支持服务。针对上述问题我们采用了数据库主备部署方案，主备都同时提供服务，每台服务的压力就会减少，提高单台数据库服务的可靠性。主库服务于全部的写操作和少量的读操作，比如说交易业务；备库服务于实时性要求不高的读取操作，比如说内部管理平台查看流水、构建各种报表业务。一旦主库发生故障无法提供服务，备库可以快速的升级为主库，继续提供服务，保证了整个数据库系统的整体可靠性。在数据库的物理文件保存方面，通过磁盘阵列技术做到磁盘容错，防止数据丢失。通过数据库主备部署和磁盘阵列方案，大大提升了数据库服务的可靠性。

三、程序设计方面容错

程序设计方面我们采用了防卫式程序设计以及降低复杂度的策略。防卫式程序设计方面首先是报文字段的检测，报文中各个参数按照预定的规则先进行检测，检测通过后再进行后续的业务处理，比如说交易金额，必须是一个正整数，比如说商户编号必须为固定 15 位长的字符串，参数检测通过后会大大降低后续业务出现不可控制的错误的概率；其次是异常处理，程序正常逻辑要处理，发生异常的情况下我们也要考虑如何处理，比如说查看商户法人证件照片，读取文件的过程中很可能会出现文件未找到的错误，一旦发生这个错误我们就要按照异常的流程处理，提示说照片文件未找到，不能进行处理；再次是进行错误信息日志记录，程序发生错误时，把相关的详细信息都记录到文件中，方便后续查找定位问题。整体的系统设计我们都采用了合理的架构模式，遵循高内聚、低耦合、单一职责、依赖倒置、接口隔离等基本的设计原则，这些都会降低整个系统的复杂性，增加系统的可靠性。

该项目于**年**月完成测试并上线，通过上述容错方案的实施，整个系统持续稳定的运行，未出现过服务中断的故障，即使系统变更上线，也可以通过集群优势，逐台设备更新上线部署，达到服务不中断的效果。后续我们又完善了 zabbix 监控系统，丰富了监控的内容，包括硬件设备监控、网络流量监控、软件进程监控、软件业务数据监控等，实现了更加全面的监控策略。错误日志记录追踪发挥了良好的作用，定期我们会排查一遍错误日志，从中分析出系统存在的问题再进行修复，进一步完善系统，进一步提高系统的可靠性。

当然，在我们设计系统和后期运行维护过程中，也陆续发现了一些问题和不足，比如说集群部署导致运维成本增加，后期我们增加了 Jenkins 自动部署的功能来解决一些问题。另一个问题就是单个服务不可用时，还是需要人工来处理，不能及时的自动新增一个服务节点上去，目前我们正在研究使用容器技术来解决该问题，等成熟稳定后也会陆续投产使用。

论企业智能运维技术与方法

智能运维（Artificial Intelligence for IT Operations, AIOps）是将人工智能应用于运维领域，基于已有的运维数据（日志数据、监控数据、应用信息等），采用机器学习方法来进一步解决自动化运维难以解决的问题。具体来说，智能运维在自动化运维的基础上，增加了一个基于机器学习的智能决策模块，控制监测系统采集运维决策所需的数据，做出智能分析与决策，并通过自动化脚本等手段去执行决策，以达到运维系统的整体目标。智能运维能够提高企业信息系统的预判能力和稳定性，降低 IT 成本，提升企业产品的竞争力。

请围绕“企业智能运维技术与方法”论题，依次从以下三个方面进行论述。

1.概要叙述你参与管理与实施的软件运维项目以及你在其中所担任的主要工作。

2.智能运维主要从效率提高、质量保障和成本管理等三个方面提升运维水平，其成熟程度可以分为尝试应用、单点应用、串联应用、能力完备和能力成熟等五个级别，请任意选择三个成熟度级别，说明其在效率提升、质量保障和成本管理等方面的特征。

3.结合你具体参与管理与实施的软件系统运维项目，举例说明如何采用智能运维技术和方法提高运维效率、保障运维质量并降低运维成本，实施效果如何。在智能运维过程中都遇到了哪些具体问题，是如何解决的。

范例

摘要部分：

年月，我所在的公司中标了**项目，内容包括**。在该项目中我担任系统架构师，负责系统的架构设计工作。本文结合该项目中个人的实践经验，介绍企业智能运维技术方法在该项目中的应用情况。我们通过引入 Kubernetes 实现微服务自动化弹性伸缩，保障在不同业务高峰时段的资源高效利用；引入 Prometheus+Grafana 实现微服务节点的观测，保障运维人员能及时收到服务预警信息；引入故障监测和自动重发机制保障系统间的数据一致性，降低了院方运维的工作量。项目于**年**月正式上线并稳定运行至今，提升了院方运维人员的工作效率，获得了用户的广泛好评。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写 280 到 300 字】

正文部分：

随着**的全面推进，**年**月，我所在的公司中标了**项目，该项目的主要内容包括**等的集成工作。

【项目背景内容可分 2 段写，第 1 段简要说明下项目的来龙去脉】

系统主要用于，包括**等功能。**负责实现**工作，以**标准。**通过**，并结合**。在该项目中，我担任系统架构师，负责系统的架构设计工作。**的业务涉及**，因此在该项目上我们引入了智能运维技术来实现整个项目的智能运维过程。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 字左右】

智能运维主要从效率提高、质量保障和成本管理三个方面提升运维水平，其成熟度可以分为尝试应用、单点应用、串联应用、能力完备和能力成熟五个级别。尝试应用级主要依赖人工操作，自动化工具初步引入但未形成闭环，效率较低，MTTR 通常在数小时以上；质量保障方面监控覆盖不完整，缺乏根因分析能力，系统可用性约为 99%；人力成本非常高，重复任务多，维护成本高且无预测性维护能力。单点应用级的自动化工具覆盖单个场景，MTTR 能缩短至 1 小时内；质量保障方面建立了基础服务管理流程，故障定位准确率较高，系统可用性可达 99.5%；运维人力成本有所降低（约 20%），但仍然存在资源浪费的问题。串联应用级通过多工具联动实现自动化运维，MTTR 一般在 10 分钟内；质量保障方面通过引入算法驱动的故障预测，系统可用性达 99.9%；总体维护成本可通过预测性维护减少资源消耗，资源利用率能提升 25% 左右。能力完备和能力成熟级的系统可用性则分别能达到 99.99% 和 99.999%。

在结合医院的实际场景并综合考虑了运维成本和实现成本后，本项目最终以串联应用级作为智能运维的成熟度建设目标，本文详细介绍如何该项目中用到的三种技术。

一、引入 Kubernetes 实现微服务自动化弹性伸缩

微服务是应对三级公立医院复杂的业务场景的必然趋势，但手工去维护这些微服务是很大的人工成本。本项目上我们基于云原生的架构去实现系统建设，各微服务用容器化方式打包成镜像，然后使用 Kubernetes 对这些镜像和副本容器的自动化部署和资源管理。但不同业务的高峰时段并不相同，比如上午时段是门诊业务的高峰期，而下午时间段则往往是住院新病人业务的高峰期，针对相应的微服务镜像分配固定节点数，容器分配固定 CPU、内存等资源，显然不利于整个服务器资源的高效利用。因此微服务节点的资源分配应当实现自动化弹性伸缩。我们在 Kubernetes 中采用 HPA+Cluster Autoscaler 的方式实现微服务资源的弹性伸缩，当资源不足（如 CPU 使用率达到预设值）时，首先通过 HPA 实现 Pod 的自动增加，而节点资源不足时 Cluster Autoscaler 会自动增加节点，以确保 Pod 调度成功。反之，当节点利用率或资源使用率低于平均阈值时，Cluster Autoscaler 和 HPA 也会进行自动缩容。据统计，在引入 Kubernetes 的弹性伸缩配置后，每天的门诊早高峰和下午的医嘱处理高峰期，服务器的 CPU 和内存使用率均为超过 60%，满足了自动化调整服务资源的需求。

二、引入 Prometheus+Grafana 实现微服务节点的观测

在引入 Kubernetes 实现微服务镜像的容器自动化部署以及容器节点和资源的自动化弹性伸缩后，发现其在容器监测上存在一定局限性：无法获取操作系统级的监测指标（如磁盘 I/O 错误率等）、无法长期存储监测数据、不具备告警功能。为此我们引入 Prometheus+Grafana 工具来实现微服务节点的观测。首先，Prometheus 定时（每 5 分钟）主动地从目标服务的 HTTP 接口拉取监控数据，通过适配器转换格式，如 Mysql 服务的 Mysqld_exporter、应用服务的 Node Exporter；其次，Prometheus 内置时序数据库 TSDB，按时间戳和 Label 存储数据；最后，由 Grafana 工具实现监控数据的可视化和告警规则的设置，并将告警消息推送到运维人员的钉钉上。Prometheus 支持指标按 metric 标识进行灵活过滤和聚合，而且 Grafana 工具也支持 20 多种图标类型，我们在现场部署了一个非常直观全面的 Grafana 看板界面，可实时观测各服务节点的资源状况。在系统的运维期间，几次因批量上传疾病报卡数据导致 Mysql 数据库 I/O 飙升，现场的运维人员都得到了及时提醒并作出了相应处理，避免了医院正常业务受到影响。

三、引入故障监测和自动重发机制保障数据一致性

医院的医疗系统中有些专业性非常强的系统，比如 LIS、PACS、输血系统等，与临床一体化系统之间业务耦合度非常高，我们采用集成平台来实现这类系统的业务集成，但某些时候因接收方服务故障或正在更新重启，往往会出现数据丢失不一致的情况，在项目运行中，多次出现因 EMR 微服务更新，医生在输血系统上开完申请单，EMR 却丢失备血医嘱的问题。为此，我们在集成平台上引入故障监测和自动重发机制。首先，我们构建了针对 ESB 消息的实时监控平台，一旦出现某个消息服务连续 20 条消息的响应都为 NACK，或一段时间内有大量格式异常的消息传入，监控平台都会发出告警，并将告警信息通过短信、钉钉的方式推送到运维人员的手机上，运维人员便可立即进行人工干预。其次，我们采用时间冗余手段，采用 ElasticSearch 存储异构的消息内容，一旦出现消息接收方无法正常处理消息的情况，集成平台每 5 分钟会进行一次消息重发，最多 3 次，也可在接收方处理能力恢复时手动重发。据院方运维人员反馈，在引入这两项机制后，大幅降低了他们在处理数据一致性问题上的工作量。

经过需求分析、系统设计、开发与测试等阶段，系统于**年**月全面上线并稳定运行至今，院方运维人员反馈相较于上线前的旧系统，运维的工作量至少降低了 50%，且系统故障率相较旧系统至少降低了 80%，实践证明，智能化运维技术可以有效缓解医院业务复杂性高、并发量高导致的运维压力。然而在项目实践中，我们也遇到了挑战，比如为了实现微服务的日志追踪，我们采用了 Jaeger 抓取+Kafka 传输+Elasticsearch 存储的方案，但在几次业务高峰期都出现了微服务容器 OOM 的问题。后续我们换成了存储成本比 Elasticsearch 低 90%且支持快速按标签过滤日志的 Loki 工具，结合 Tempo（Grafana 的追踪后端）解决了该问题。智能运维技术是医疗信息化发展的关键技术领域，我们团队也将在未来多学习相关方面的知识，为“健康中国”战略提供坚实的技术支持。

论软件的系统测试及其应用

软件测试是软件交付客户前必须要完成的重要步骤之一，目前仍是发现软件错误（缺陷）的主要手段。系统测试是将已经确认的软件、计算机硬件、外设、网络等其他元素结合在一起，针对整个系统进行的测试，目的是验证系统是否满足了需求规格的定义，找出与需求规格不符或与之矛盾的地方，从而提出更加完善的方案。系统测试的主要内容包括功能性测试、健壮性测试、性能测试、用户界面测试、安全性测试、安装与反安装测试等。

请围绕“软件的系统测试及其应用”论题，依次从以下三个方面进行论述。

1.概要叙述你参与管理和开发的软件项目以及你在其中所担任的主要工作。

2.详细论述软件的系统测试的主要活动及其所包含的主要内容，并说明功能性测试和性能测试的主要的目的。

3.结合你具体参与管理和开发的实际项目，概要叙述如何采用软件的系统测试方法进行系统测试，说明具体实施过程以及应用效果。

范例

摘要部分：

年月，本公司拟搭建**平台，实现**，主要包括工序母版管理、工序卡实例生成、生产信息管理、产线信息追踪等核心模块。我在该项目中担任系统架构师，主要负责该系统的架构设计工作。本文以卫星产线信息化平台为例，主要讨论了系统测试技术在该项目中的具体应用。通过功能测试确保系统实现了全部的需求功能点及功能的正确性；通过压力测试，确保系统在大面积铺开使用时可以稳定运行；通过并发测试，保障系统在高并发情况下的正常响应。我们通过利用恰当的系统测试技术与工具，使系统测试更加精准可靠，大大提升了系统上线后的稳定性。最终项目研发顺利，系统使用功能性能正常，并获得领导和同事的一致好评。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写 280 到 300 字】

正文部分：

本人就职于**公司，本公司主要从事**。由于**，经营层决定自研**平台。【项目背景内容可分 2 段写，第 1 段简要说明下项目的来龙去脉】

项目一期于**年**月开始建设，计划用时 2 年，完成**工作。本人有幸担任系统架构师，主要负责该系统的架构设计工作。平台主要包括**等核心模块，实现**工作。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 字左右】

由于本项目涉及多个技术部门的核心业务，平台上线后的稳定性将直接影响卫星脉动生产线的生产效率与生产追踪。因此，如何提升系统的稳定性及可靠性，针对拟开发平台的系统测试显得至关重要。系统测试主要以发现软件错误为核心，验证开发软件在各方面是否满足需求。主要内容包括功能测试、性能测试、健壮性测试、用户界面测试、安全性测试、安装和反安装测试等。其中，最重要的工作是进行功能测试和性能测试。功能测试主要采取黑盒测试方式，主要目的是检查功能是否按照 SRS 的要求正常使用，软件是否能够恰当地接收输入数据并产生正确的输出信息，软件运行过程中能否保持外部信息的完整性。性能测试则主要验证软件系统在承担一定负载的情况下所表现出来的特性是否符合产品设定的需要，主要指标包括响应时间、吞吐量、并发用户数和资源利用率等。性能测试主要目的验证软件系统是否能达到用户提出的性能指标，同时发现软件系统中存在的性能瓶颈等问题。

考虑到卫星产线信息化平台的实际使用与部署情况，我主要对系统进行了详细的功能测试以及性能测试中的压力测试、并发测试等几个方面。

一、功能测试

系统功能测试我们主要用以验证开发软件是否覆盖了用户提出的所有功能需求以及功能的正确性。首先，我们根据需求分析文档，对每个功能模块进行了细化分解，建立功能测试用例。例如，工步信息上报模块，按使用角色包括上报人员与确认人员两个角色，我们根据角色划分的具体工作拆开编写测试用例，其中元件拍摄记录、上报信息填写等归属于上报人员角色，逐项确认、验收上报确认等归属于确认人员角色。子模块继续拆分，例如元件拍摄记录可拆分为唤醒相机拍照，系统图像选取、图像判定等具体功能点，再对功能点继续拆分就是每个业务功能中所包含的具体数据输入、保存操作、确定操作等最小功能点。经过这种自上而下的梳理、拆分以及逐项测试，确保功能不缺失。再然后，针对具体的数据输入项，为保证数据输入的合理有效，我们又针对特殊字段进行了等价类划分的用例测试。例如，对器件重量，尺寸，电能等输入项进行了有效类和无效类划分。同时，结合边界值分析方法，通过测试确保了系统输入数据的正确有效。

二、压力测试

压力测试我们主要用于测试系统能正常运行的最大服务级别，评估是否在后续实际环境中能确保服务的稳定性。由于本平台系统管理业务规模较大，并结合公司发展规划，预测短时间内系统使用量会持续大幅增加，因此服务器在高压环境下的运行情况至关重要。为测试服务器是否能够承受住相应压力，我们根据系统部署方案和测试用例，利用 LoadRuner 测试工具在服务器上进行压力模拟测试，评估压力峰值，并观测记录 CPU、内存等在不同压力下的使用情况。为了充分利用服务器的同时又可以保证系统的稳定运行，我们设定 CPU 和内存到达 80% 为最高上限，达到最高上限即为可承受的最大压力值。在压力测试后，针对没有达到预计标准的情况，我们首先进行了程序上的优化，提高 CPU 的使用率和降低对内存的消耗，关于有些优化不了的情况，我们考虑扩充了服务器的相关配置，最后保证了系统压力性能的要求。

三、并发测试

并发测试我们主要用于测试系统可处理的同时在线的最大用户数。产线信息化平台采用 B/S 架构，前后端服务器统一部署，用户主要采用电脑终端与移动设备(主要以 PAD 为主)。针对系统在大面积铺开使用后，高并发情况下系统是否能保持稳定、是否能保证响应正常等问题，我们进行了有效的并发测试。针对并发测试，我们编写了详细的测试用例，选取并发场景较多的模块进行了重点测试，例如用户信息校验模块、元件拍摄与判定模块等。我们同样利用 LoadRuner 软件，采用地毯式的逐渐增加阈值量的方式进行测试，例如，对用户信息校验模块，我们模拟了 500、800、1100、1400 等不同阈值，测试在不同并发情况下，系统的响应速度情况。如果测试过程中出现问题，我们就针对问题进行调试优化，通过多线程、扩展服务器等方式，满足系统的并发要求，保证了平台的高可用性与用户的使用效果。

通过上述系统测试技术在本项目中的适当应用，我们圆满的完成了项目的测试任务，保障了系统功能与各方面性能达到用户要求。最终项目历时**年**个月，于**年**月初完成验收测试，**月正式投入使用，提前完成研发目标。在系统投入使用后，得益于系统测试较为完备可靠，系统稳定性较强，基本实现了卫星脉动生产线的信息化工作，获得领导和各部门同事的一致好评。

事后复盘整个系统测试过程，发现虽然整体上比较成功，但也存在着一些不足之处。例如关于签字报告生成模块，由于用户在开发过程中提出过需求变更，但在开发团队修改功能后，未再次组织进行详细的响应测试，导致用户在系统上线后实际使用该功能时，发现页面加载时间较长，后重新组织人员对该模块进行测试并优化，使响应时间优于 500ms，解决了该问题，最后获得用户的认可。后续再次出现需求变更情况，一定要谨记针对修改后功能再次进行详细测试，避免漏洞的出现。接下来，作为系统架构师，我会总结现阶段的经验教训，在后续平台二期建设中，不断思考和改进系统测试中的不足，在后续项目中充分发挥系统测试技术的更大作用。